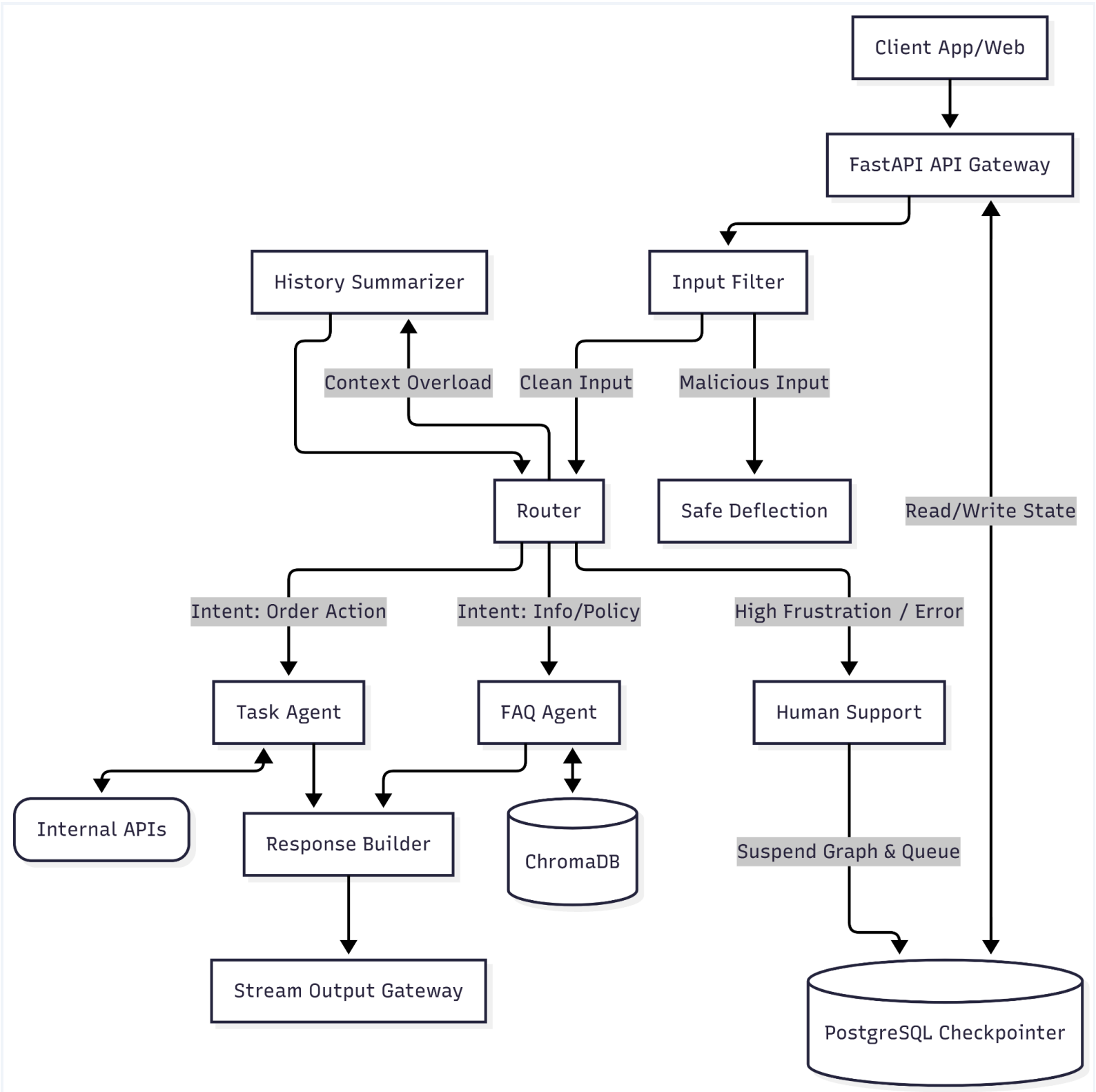
AI Support Agent Architecture -Taking Quick Commerce as an example

Below is the visual representation of the LangGraph and infrastructure flow.



Node Specifications

- **1. Input Guardrail Node (The Shield):** A fast, low-latency evaluator that intercepts the raw input before it reaches the main LLM. It blocks prompt injections, jailbreaks, and sanitizes PII (masking phone numbers or payment details).
- **2. Router Node (The Brain):** The primary decision-maker powered by Gemini. It extracts the order_id, analyzes the customer's sentiment, normalizes code-switching (e.g., seamlessly understanding a mix of English and Hindi), and classifies the exact intent to route the state to the correct specialized worker.
- **3a. OMS Agent Node (The Doer):** A tool-calling node strictly bound to internal backend APIs. It handles transactional requests like fetching ETAs, verifying stock, or issuing micro-refunds based on strict business logic.
- **3b. Policy RAG Node (The Librarian):** Handles non-transactional queries. It queries ChromaDB to retrieve shipping policies, return guidelines, or general FAQs, returning exact document chunks to ground the final answer.
- **3c. Memory Summarizer Node (The Optimizer):** Triggered conditionally when the conversational thread gets too long. It runs a background text summarizer over older messages, compressing the history to maintain a lean context window and avoid token limit crashes.
- **3d. Human Handoff Node (The Safety Net):** If the Triage node detects high anger, or the OMS node hits a critical error, the graph suspends execution. The state is parked in the database, and the ticket is flagged for human intervention.
- **4. Response Synthesizer Node:** Takes the raw JSON from the OMS or the text chunks from RAG and uses Gemini to craft a polite, empathetic, and perfectly formatted response. It acts as the final anti-hallucination check, ensuring the bot only promises what the data confirms.

This is Production-Ready

1. Scalability

The entire conversational state is pushed out of system memory and managed by a robust PostgreSQL checkpointer. This renders your FastAPI application completely stateless, allowing you to horizontally scale your worker nodes across multiple AWS EC2 instances. When the dinner-rush traffic hits, the architecture effortlessly scales out to handle the concurrent connections.

2. State Durability & Concurrency

By utilizing LangGraph's checkpointer backed by PostgreSQL (ideally with a connection pooler like PgBouncer), concurrent reads and writes are handled safely. If an EC2 instance goes down mid-conversation, the state is preserved in the database, and another instance picks up the thread exactly where it left off.

3. Graceful Degradation & Fault Tolerance

In q-commerce, internal APIs will occasionally spike in latency or timeout. The OMS Agent Node is designed with explicit try/catch tool boundaries. If an internal service fails, LangGraph catches the exception and routes the flow to the Synthesizer with an error state, triggering a polite fallback message rather than crashing the thread.

4. Context and Token Efficiency

Long, looping conversations drain tokens and slow down response times. The inclusion of the Memory Summarizer node ensures the LLM never chokes on its own context window, keeping latency low even on the 50th message of a difficult support ticket.